Table of contents

Part 1 — Foundation: your Raspberry Pi and your Tailscale network **1**
Why Tailscale is the spine of everything . . . 1
Prerequisites . . . 2
Step 1 — Flash a headless 64-bit Linux with key-based SSH . . . 2
Step 2 — First boot, then install Docker . . . 3
Step 3 — Put the Pi on your tailnet . . . 3
Step 4 — Name the Pi `homelab` and turn on MagicDNS . . . 3
Step 5 — Install Tailscale on your laptop . . . 4
Step 6 — Install Tailscale on your iPhone . . . 4
Step 7 — Verify the mesh . . . 4
Recap . . . 4

Part 1 — Foundation: your Raspberry Pi and your Tailscale network

> **The payoff of this part:** an always-on Raspberry Pi that's ready to host everything in this series — running Docker, hardened SSH, and joined to a private Tailscale network — plus your laptop and iPhone on that same network, so every machine can reach every other by name, from anywhere, with nothing exposed to the public internet.

This is the groundwork the rest of the book stands on. We don't install any "app" here — instead we build the platform: a Pi that's online, secured, and addressable, and a **tailnet** (your private Tailscale network) that stitches your devices together. Every later part — Audiobookshelf, Pi-hole, the dashboard, the VPN — simply plugs into what you set up now.

Why Tailscale is the spine of everything

Tailscale is a zero-config mesh VPN built on WireGuard. Once installed on your devices and signed into one account, each device gets a stable private address (a `100.x.y.z`) and can reach the others directly — on any network, with no port forwarding and nothing open to the internet.

Why this instead of port forwarding:

- Port forwarding needs a public IP, router configuration, dynamic DNS if your ISP rotates addresses, and it exposes your services to the entire internet.
- A relay like Cloudflare Tunnel avoids port forwarding but proxies all your traffic through a third party.
- Tailscale punches out from both ends and builds a direct, encrypted WireGuard tunnel. Your Pi never accepts an inbound connection from the public internet — only devices on *your* tailnet can talk to it.

That last point is why this guide never tells you to open a router port: the homelab is private by default and works even on locked-down corporate Wi-Fi.

Prerequisites

- A **Raspberry Pi 4 or 5** with 4 GB+ RAM, a power supply, and a microSD card (16 GB+). The Pi will be your always-on server. *This guide was built on a **Raspberry Pi 4 Model B (8 GB)** with a **32 GB microSD card**, in an **Argon ONE M.2 Aluminum case**, running **Ubuntu Server 26.04 LTS (64-bit), headless** — comfortable headroom, but not a minimum. Raspberry Pi OS Lite works identically.*
- A **laptop/desktop** (Linux assumed; the commands use `apt`/`systemd`) to flash the card and to use as a client.
- An **iPhone** (the apps used later also exist for Android; commands are identical).
- A **free Tailscale account** — sign up with Google/GitHub/Microsoft or email at https://tailscale.com. One account, signed into on every device, is the only "server" you need.

Step 1 — Flash a headless 64-bit Linux with key-based SSH

Install **Raspberry Pi Imager** on your laptop. This guide's reference build runs **Ubuntu Server 26.04 LTS (64-bit)** — in Imager, choose **Other general-purpose OS → Ubuntu → Ubuntu Server 26.04 LTS (64-bit)**. (**Raspberry Pi OS Lite (64-bit)** works exactly the same way and the rest of the guide is identical — pick either; both are headless, just SSH and a shell, which is all a server needs.)

Before writing the card, open Imager's settings (the gear / **Edit settings**):

- Set a **hostname** (e.g. `homelab`) and your **username**.
- Under **Services**, enable **SSH → "Allow public-key authentication only"**, and paste your laptop's public key. If you don't have one yet, generate it first:

```
ssh-keygen -t ed25519        # then paste the contents of
~/.ssh/id_ed25519.pub
```

This bakes key-based login in from first boot and disables password SSH — the single most important hardening step for a box that's always on.

> **!** Remoting safety for an always-on server
>
> You'll SSH into this Pi for the rest of the series, so set it up safely once:
>
> - **Key-based auth only — no password login.** The Imager option above handles it; on an already-running Pi you'd `ssh-copy-id you@homelab`, then set `PasswordAuthentication no` in `/etc/ssh/sshd_config` and `sudo systemctl restart ssh`.
> - **Reach it over Tailscale, never the public internet.** Once the Pi is on your tailnet (Step 3) you SSH to it as `you@homelab` from anywhere — there is never a reason to port-forward SSH (or anything) on your router. An open port 22 draws constant brute-force traffic; Tailscale sidesteps it entirely.
> - **Keep it patched:** `sudo apt update && sudo apt upgrade -y` periodically, or enable `unattended-upgrades`.

Write the card, put it in the Pi, and power on.

Step 2 — First boot, then install Docker

Find the Pi on your LAN and SSH in (`ssh you@<pi-lan-ip>`, or `ssh you@homelab.local` if your network supports mDNS). Update it and install Docker, which every later part uses to run its services:

```
# Update the base image
sudo apt update && sudo apt upgrade -y

# Docker - the official one-line installer
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER
newgrp docker        # apply the group change in this shell
```

> **Why Docker**
>
> Every service in this series (Audiobookshelf, Pi-hole, the dashboard, the reverse proxy) ships as a Docker container. Running them in containers keeps their dependencies off your host, makes upgrades a single `docker pull`, and lets each one live in its own tidy folder with a `compose.yaml`. Installing it once here means every later part just runs `docker compose up`.

Step 3 — Put the Pi on your tailnet

```
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up
```

`tailscale up` prints a URL — open it, sign in, and the Pi joins your tailnet. Check its address:

```
tailscale ip -4      # something like 100.x.y.z
```

That `100.x.y.z` is how every other device will reach the Pi. You won't have to memorize it, though — MagicDNS (next step) gives it a name.

Step 4 — Name the Pi `homelab` and turn on MagicDNS

MagicDNS gives every device on your tailnet a stable name, so you can type `homelab` instead of an IP. In the Tailscale admin console (https://login.tailscale.com):

1. **DNS** page → confirm **MagicDNS** is enabled (it's on by default for tailnets created after late 2022; the button reads "Disable MagicDNS" when on). Note your **tailnet name** here — something like `tail1a2b3.ts.net`; wherever this guide writes `your-tailnet.ts.net`, substitute it.

2. **Machines** page → the ⋯ menu on the Pi's row → **Edit machine name** → set it to `homelab`.

Now the Pi answers to `homelab` (and its full name `homelab.your-tailnet.ts.net`) from any device on the tailnet. Because MagicDNS installs your tailnet name as a **search domain**, the short name `homelab` usually works on its own; the full `homelab.your-tailnet.ts.net` is the always-reliable fallback.

Step 5 — Install Tailscale on your laptop

```
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up        # sign in with the SAME account
```

Your tailnet is one shared namespace — signing in with the same account simply adds the laptop as another peer. From now on you can `ssh you@homelab` from the laptop over the tailnet, anywhere.

Step 6 — Install Tailscale on your iPhone

Install **Tailscale** from the App Store and sign in with the same account. You should see `homelab` (and your laptop) listed as peers. That's it — the phone is now on the mesh and can reach the Pi by name, on Wi-Fi or cellular.

Step 7 — Verify the mesh

From the Pi (or your laptop):

```
tailscale status         # lists every device with a green marker when online
ping homelab             # from the laptop/phone - resolves via MagicDNS
```

If `homelab` resolves and `tailscale status` shows your laptop and phone online, the foundation is done.

> **A naming note you'll be glad to know later**
>
> `nslookup homelab` and `host homelab` can *fail* on macOS even when everything works — those tools bypass the resolver MagicDNS hooks into. Test with `ping homelab` or a browser instead, and keep `homelab.your-tailnet.ts.net` as the guaranteed-resolvable fallback. This trips people up; it's not a broken setup.

Recap

- **Flashed** a headless 64-bit Linux (Ubuntu Server 26.04 LTS) with key-only SSH (no passwords, no open ports).
- **Installed Docker** on the Pi — the runtime for every later service.

- **Built your tailnet:** the Pi, your laptop, and your iPhone all on one private WireGuard mesh, signed into one account.
- **Named the Pi `homelab`** via MagicDNS, so everything is reachable by name.

The platform is ready. In Part 2 we put it to work: ripping your Assimil discs and serving them from the Pi to your iPhone over the tailnet you just built.